Individual Assignment: University Course Management System

Due Date: 5th April 2026 **Marks: 20**

Objective: This assignment aims to assess your understanding and ability to apply Object-Oriented Programming (OOP) principles, Generics, and Abstract Data Type (ADT) design in Java to build a functional application. You will create a system to manage different types of courses and instructors in a university.

Problem Statement: A university requires a software system to manage its course offerings. The system must handle different types of courses (e.g., Lecture-based and Lab-based) and store details about the instructors assigned to them. The university requires a flexible system that uses generics to handle different types of data and an ADT to define the core contract for all courses.

Tasks:

There are four tasks in total, you are required to do all the task by 5th April 2026 and submit according to the submission guidelines given at the end. In week 5, during tutorial and lab hours, tutorial instructor and lab demonstrators of respective OCC will conduct viva session, they will take viva based on your submission and the marking will be done based on this Face-to-face viva.

Task 1. **Define an interface `Course` (ADT) that has the following methods. For each method, you must provide comments stating its purpose, and parameters requirement (if any).**

- `getCourseCode()`: Returns the course's unique code.
- `getCourseTitle()`: Returns the course's title.
- `calculateTotalWorkload()`: Returns the total workload hours for the course per semester.
- `getInstructorName()`: Returns the name of the assigned instructor.
- `printCourseDetails()`: Prints all relevant information about the course.

Task 2. The following are incomplete classes for `LectureCourse` and `LabCourse`. Study their structure and then answer the multiple choice questions below.

- **LectureCourse Class:**

```
public class LectureCourse implements Course {
    private String courseCode;
    private String courseTitle;
    private String instructorName;
    private int lectureHoursPerWeek;

    public LectureCourse(String courseCode, String courseTitle, String instructorName, int
    lectureHoursPerWeek)
   {
        this.courseCode = courseCode;
        this.courseTitle = courseTitle;
        this.instructorName = instructorName;
        this.lectureHoursPerWeek = lectureHoursPerWeek;
    }

    @Override
    public String getCourseCode() {
        return courseCode;
    }

    @Override
    public String getCourseTitle() {
        return courseTitle;
    }

    @Override
    public int calculateTotalWorkload() {
    // Workload is lecture hours per week * number of weeks in a semester (assume 14
    weeks)
        return lectureHoursPerWeek * 14;
    }

    @Override
    public String getInstructorName() {
        return instructorName;
    }

    @Override
    public void printCourseDetails() {
        System.out.println("Lecture Course: " + courseTitle + " (" + courseCode + ")");
        System.out.println("  Instructor: " + instructorName);
```

```
        System.out.println("  Lecture Hours/Week: " + lectureHoursPerWeek);
        System.out.println("  Total Workload (Semester): " + calculateTotalWorkload() + "
        hours");
    }
}
```

- **`LabCourse` Class:**

```
public class LabCourse implements Course {
    private String courseCode;
    private String courseTitle;
    private String instructorName;
    private double labHoursPerWeek;
    private int numberOfSessions;

    public LabCourse(String courseCode, String courseTitle, String instructorName, double
labHoursPerWeek, int numberOfSessions) {
        this.courseCode = courseCode;
        this.courseTitle = courseTitle;
        this.instructorName = instructorName;
        this.labHoursPerWeek = labHoursPerWeek;
        this.numberOfSessions = numberOfSessions;
    }

    @Override
    public String getCourseCode() {
        return courseCode;
    }

    @Override
    public String getCourseTitle() {
        return courseTitle;
    }

    // --- MISSING METHOD IMPLEMENTATIONS ---
    // TODO: Implement getInstructorName()
    // TODO: Implement calculateTotalWorkload()
    // TODO: Implement printCourseDetails()
    // ----------------------------------------------
}
```

Based on the provided code, answer the following multiple-choice questions:

1. **Given the following code:**

```
LectureCourse lecCourse = new LectureCourse("CS101", "Intro to Programming", "Dr. Smith", 3);

System.out.println(lecCourse.calculateTotalWorkload());
```

What will be the output?

a) 3

b) 14

c) 42

d) 17

2. **Given the following code:**

```
LabCourse labCourse = new LabCourse("CS201", "Data Structures Lab", "Prof. Jones", 2.5, 2);
System.out.println(labCourse.getCourseCode());
```

What will be the output?

a) CS201
b) Data Structures Lab
c) Prof. Jones
d) 2.5

3. **What will be the output of the following code snippet?**

```
LectureCourse lec1 = new LectureCourse("CS101", "Intro to Programming", "Dr. Smith", 3);
LectureCourse lec2 = new LectureCourse("CS102", "OOP", "Prof. Lee", 4);
System.out.println(lec2.calculateTotalWorkload());
```

a) 56
b) 42
c) 3
d) 4

4. **Given the following code, what is the expected output?**

```
LabCourse lab1 = new LabCourse("CS201", "Data Structures Lab", "Prof. Jones", 2.5, 2);
LabCourse lab2 = new LabCourse("CS202", "Algorithms Lab", "Dr. Brown", 2.0, 1);
System.out.println(lab1.getCourseTitle());
```

a) CS201
b) Data Structures Lab
c) Prof. Jones
d) 2.5

5. **Considering the `LectureCourse` class, what would be the result of the following code?**

```
LectureCourse lec = new LectureCourse("CS101", "Intro to Programming", "Dr. Smith", 3);
lec.printCourseDetails();
```

a) A compilation error.
b) A runtime error.
c) The details of the lecture course will be printed to the console.
d) Nothing will be printed.

6. **What is the primary difference between how a `LectureCourse` and a `LabCourse` calculate their total workload?**

a) `LabCourse` uses hours per week * sessions, while `LectureCourse` uses hours per week.
b) `LabCourse` uses a `double` for hours, while `LectureCourse` uses an `int`.
c) `LectureCourse` uses weeks in a semester, while `LabCourse` does not.
d) `LabCourse` does not have a `calculateTotalWorkload()` method.

7. **If we add a new type of course, `OnlineCourse`, which has `videoHours` and `quizCount` as attributes, what method from the `Course` interface would need to be implemented in a unique way for this new class?**

a) `getCourseCode()`
b) `getCourseTitle()`
c) `calculateTotalWorkload()`
d) `printCourseDetails()`

8. **What is the purpose of the `@Override` annotation used in the `LectureCourse` class?**

a) It tells the compiler to ignore the method.
b) It tells the compiler that this method is intended to override a method in a superclass or interface.
c) It is required for all methods in an interface.
d) It is used to define a new method.

Task 3. Define a generic class `CourseManager<T extends Course>` that manages a collection of `Course` objects using `ArrayList<T>`. The `CourseManager` class should support the following operations:

- `addCourse(T course)`: Adds a course to the manager.
- `removeCourse(String courseCode)`: Removes a course by its unique code.
- `getCourseWithHighestWorkload()`: Retrieves and returns the course with the highest total workload.
- `sortCoursesByWorkload()`: Sorts the courses by their total workload in ascending order.
- `printAllCourses()`: Prints the details of all courses in the collection.

Task 4. Implementation and Testing

Complete the `LabCourse` class by implementing the missing methods (`getInstructorName()`, `calculateTotalWorkload()`, and `printCourseDetails()`). The total workload for a lab course should be calculated as `labHoursPerWeek * numberOfSessions * 14` (assuming 14 weeks in a semester).

Implement the `Course` interface (if not already done) and the `CourseManager` class.

Create a `Main` class to test your system. In the `Main` class, perform the following operations in sequence:

1. Create at least two `LectureCourse` objects and two `LabCourse` objects with sample data of your choice.
2. Create a `CourseManager` instance.
3. Add all the created courses to the manager.
4. Print the details of the course with the highest total workload.
5. Sort the courses by their total workload and print all course details.
6. Remove a course by its course code and print the remaining course details to confirm it was removed.

Submission Guidelines:

- Submit a single zip file containing all your Java source code files (`.java`) and word File with answers to Multiple choice questions.

- Ensure your code is well-commented and follows standard Java naming conventions.

- Your code must be compilable and runnable with no errors.

Marking Scheme:

- **Task 1 (ADT Design):** 4
  - Correct interface definition.
  - Clear and appropriate code comments.
- **Task 2 (Code Analysis):** 4
  - Correct answers to multiple-choice questions.
- **Task 3 (Generic Class):** 6
  - Correct use of generics (`<T extends Course>`).
  - Proper implementation of `ArrayList` operations.
  - Correct logic for `getCourseWithHighestWorkload` and `sortCoursesByWorkload`.
- **Task 4 (Implementation & Testing):** 6
  - Correct completion of `LabCourse` class.
  - Correct implementation of `CourseManager`.
  - Correct logic and output in `Main` class demonstrating all required operations.
  - Code quality, readability, and comments.